

Adopet: Ingeniería de Requisitos y Arquitectura por Capas para un Sistema de Gestión de Adopción de Mascotas

Julián Darío Romero Buitrago Juan Esteban Moreno Durán Yuber Alejandro Bohorquez Roa

Código: 20232020240

Código: 20232020107

Código: 20231020195

Ingeniería de Sistemas
Universidad Distrital Francisco José de Caldas
Bogotá D.C., Colombia

Resumen—Este documento presenta el levantamiento, especificación y análisis de requerimientos para Adopet, un sistema de software orientado a gestionar el proceso de adopción de mascotas. La solución centraliza la información relacionada con mascotas, adoptantes, solicitudes y seguimiento de los procesos de adopción.

El sistema contempla dos tipos principales de usuarios: adoptantes y administradores. Se definen requerimientos funcionales y no funcionales, reglas de negocio, casos de uso, historias de usuario y criterios de aceptación. Adicionalmente, se analizan tres enfoques ágiles: Scrum, Kanban y Extreme Programming (XP), seleccionando Scrum como marco principal para el desarrollo.

Finalmente, se establece una arquitectura por capas de tres niveles, complementada con el patrón Modelo–Vista–Controlador (MVC), con el propósito de facilitar la separación de responsabilidades, mantenimiento, pruebas y evolución del sistema.

Index Terms—Adopet, adopción de mascotas, ingeniería de software, requerimientos, historias de usuario, Scrum, Kanban, Extreme Programming, arquitectura por capas, MVC.

I. INTRODUCCIÓN

La adopción de mascotas requiere administrar información sobre animales disponibles, personas interesadas, solicitudes y seguimiento de cada proceso. Cuando esta información se gestiona mediante mecanismos dispersos, pueden aparecer problemas de actualización, trazabilidad, control y comunicación.

Adopet se propone como un sistema de información que centraliza y organiza este proceso. Los adoptantes podrán consultar mascotas disponibles, conocer sus características, registrar solicitudes y consultar su estado. Los administradores podrán gestionar mascotas, revisar solicitudes, actualizar estados y consultar el historial.

El documento integra el levantamiento de información, la especificación de requerimientos funcionales y no funcionales, reglas de negocio, análisis de requerimientos, casos de uso, historias de usuario, metodologías ágiles, arquitectura y trazabilidad.

Para la especificación de requisitos se toma como referencia ISO/IEC/IEEE 29148:2018 [1].

II. OBJETIVOS

II-A. Objetivo general

Definir, analizar y documentar los requerimientos y la arquitectura de **Adopet**, con el fin de establecer una base técnica para desarrollar un sistema que gestione el proceso de adopción de mascotas.

II-B. Objetivos específicos

- Identificar las necesidades de adoptantes y administradores.
- Definir el alcance y los actores del sistema.
- Establecer requerimientos funcionales y no funcionales verificables.
- Transformar las necesidades identificadas en historias de usuario y casos de uso.
- Analizar y comparar enfoques ágiles adecuados para un equipo de tres integrantes.
- Seleccionar y definir la implementación de un marco ágil para el proyecto.
- Proponer una arquitectura por capas que facilite mantenimiento, pruebas y evolución.

III. DESCRIPCIÓN Y ALCANCE DEL SISTEMA

Adopet cubrirá el proceso de adopción desde el registro y publicación de mascotas hasta la gestión y seguimiento de las solicitudes.

III-A. Usuarios

- **Adoptante:** persona interesada en adoptar una mascota.
- **Administrador:** usuario responsable de gestionar mascotas, usuarios y solicitudes.

III-B. Alcance funcional

El sistema permitirá:

- Registrar e iniciar sesión.
- Gestionar el perfil del adoptante.
- Registrar, actualizar y desactivar mascotas.
- Consultar, buscar y filtrar mascotas.
- Visualizar información detallada de las mascotas.
- Registrar y consultar solicitudes de adopción.

- Revisar, aprobar y rechazar solicitudes.
- Realizar seguimiento al estado de las solicitudes.
- Actualizar la disponibilidad de las mascotas.
- Consultar el historial de solicitudes y adopciones.

III-C. Fuera de alcance inicial

No se contemplan inicialmente pagos, transporte, servicios veterinarios, integraciones gubernamentales, aplicación móvil ni integración con redes sociales.

Estas funciones podrán considerarse en futuras versiones del sistema.

IV. LEVANTAMIENTO DE INFORMACIÓN

IV-A. Problema identificado

El proceso de adopción puede involucrar registros manuales, publicaciones independientes y seguimiento mediante diferentes medios. Esto puede generar información desactualizada, duplicidad, dificultad para conocer el estado de una solicitud y pérdida de trazabilidad.

Adopet busca solucionar estas dificultades mediante una plataforma centralizada que permita gestionar la información relacionada con las mascotas, los adoptantes y las solicitudes de adopción.

IV-B. Técnicas propuestas

- **Entrevistas:** identificar necesidades y dificultades de adoptantes y responsables del proceso de adopción.
- **Observación:** analizar el flujo real de registro, solicitud, evaluación y cierre de una adopción.
- **Revisión documental:** estudiar formularios, registros y datos utilizados durante el proceso.
- **Historias de usuario:** convertir necesidades identificadas en funcionalidades expresadas desde la perspectiva del usuario.

IV-C. Información a recolectar

- Datos de identificación y contacto del adoptante.
- Características, estado y disponibilidad de cada mascota.
- Información necesaria para una solicitud de adopción.
- Estados y observaciones del proceso.
- Criterios de aprobación o rechazo definidos por la organización.
- Necesidades de consulta y reportes administrativos.

V. ACTORES DEL SISTEMA

V-A. Adoptante

El adoptante es la persona interesada en realizar una adopción. Podrá registrarse, autenticarse, administrar su perfil, consultar mascotas, buscar y filtrar resultados, visualizar detalles, registrar solicitudes de adopción y consultar el seguimiento de sus procesos.

V-B. Administrador

El administrador será responsable de gestionar la información del sistema. Podrá autenticarse, registrar y actualizar mascotas, consultar adoptantes, revisar solicitudes, aprobar o rechazar procesos, actualizar estados y consultar el historial.

VI. REQUERIMIENTOS

VI-A. Requerimientos funcionales

- **RF-01 – Registro de adoptante:** El sistema deberá permitir crear una cuenta con datos personales, información de contacto y contraseña.
- **RF-02 – Inicio de sesión:** El sistema deberá autenticar a adoptantes y administradores mediante credenciales válidas.
- **RF-03 – Gestión de perfil:** El adoptante deberá consultar y actualizar su información personal.
- **RF-04 – Registro de mascotas:** El administrador deberá registrar nombre, especie, raza, sexo, edad, tamaño, descripción, estado de salud, imagen y disponibilidad.
- **RF-05 – Actualización de mascotas:** El administrador deberá modificar la información registrada de una mascota.
- **RF-06 – Desactivación de mascotas:** El administrador deberá retirar una mascota de las publicaciones activas sin eliminar su historial.
- **RF-07 – Consulta de mascotas:** El adoptante deberá consultar las mascotas disponibles.
- **RF-08 – Búsqueda y filtrado:** El sistema deberá permitir filtrar por especie, raza, sexo, edad, tamaño y disponibilidad.
- **RF-09 – Detalle de mascota:** El sistema deberá mostrar información detallada de la mascota seleccionada.
- **RF-10 – Solicitud de adopción:** El adoptante deberá registrar una solicitud de adopción asociada a una mascota disponible.
- **RF-11 – Validación de solicitud:** El sistema deberá impedir solicitudes sobre mascotas que no estén disponibles.
- **RF-12 – Consulta de solicitudes:** El adoptante deberá consultar sus solicitudes y conocer su estado.
- **RF-13 – Revisión administrativa:** El administrador deberá consultar las solicitudes recibidas.
- **RF-14 – Actualización de solicitud:** El administrador deberá establecer estados como Pendiente, En revisión, Aprobada, Rechazada y Finalizada.
- **RF-15 – Seguimiento:** El sistema deberá registrar y permitir consultar cambios relevantes del proceso de adopción.
- **RF-16 – Actualización de disponibilidad:** Al finalizar una adopción, la mascota deberá pasar al estado Adoptada.
- **RF-17 – Historial:** El sistema deberá conservar el historial de solicitudes y adopciones.
- **RF-18 – Gestión de usuarios:** El administrador deberá consultar y gestionar las cuentas de adoptantes.
- **RF-19 – Recuperación de contraseña:** El sistema deberá permitir a adoptantes y administradores restablecer su contraseña mediante un mecanismo de verificación (p. ej. correo electrónico).
- **RF-20 – Notificaciones al adoptante:** El sistema deberá notificar al adoptante los cambios de estado relevantes de sus solicitudes de adopción.

VI-B. Requerimientos no funcionales

- **RNF-01 – Usabilidad:** La interfaz deberá ser clara, consistente e intuitiva.
- **RNF-02 – Rendimiento:** Las consultas comunes deberán responder en menos de 3 segundos bajo condiciones normales de infraestructura.
- **RNF-03 – Seguridad:** Las contraseñas deberán almacenarse mediante un mecanismo de hash seguro y nunca en texto plano.
- **RNF-04 – Control de acceso:** Las funcionalidades deberán restringirse según el rol del usuario (Adoptante o Administrador).
- **RNF-05 – Integridad:** Las relaciones entre adoptantes, mascotas y solicitudes deberán mantener consistencia referencial.
- **RNF-06 – Disponibilidad:** El sistema deberá estar disponible durante los periodos definidos para su utilización.
- **RNF-07 – Mantenibilidad:** La solución deberá mantener una separación clara de responsabilidades para facilitar cambios y correcciones.
- **RNF-08 – Escalabilidad:** La arquitectura deberá permitir agregar módulos sin rediseñar completamente el sistema.
- **RNF-09 – Compatibilidad:** La aplicación web deberá funcionar en navegadores modernos.
- **RNF-10 – Respaldo:** La información deberá poder respaldarse periódicamente.

VI-C. Reglas de negocio

Cuadro I
REGLAS DE NEGOCIO

ID	Descripción
RN-01	Una mascota solo estará disponible cuando su estado sea Disponible.
RN-02	Solo Adoptantes o Administradores autenticados podrán operar el sistema.
RN-03	No se podrá solicitar una mascota Adoptada o No disponible.
RN-04	Cada solicitud debe asociarse a un Adoptante y una mascota existentes.
RN-05	Solo el Administrador podrá aprobar o rechazar solicitudes.
RN-06	Una adopción finalizada cambia la mascota a estado Adoptada.
RN-07	Los procesos finalizados se conservan como historial.
RN-08	Cada actor solo accede a las funciones permitidas por su rol.

VII. ANÁLISIS DE REQUERIMIENTOS

El análisis de requerimientos permite relacionar las necesidades identificadas con las funcionalidades que serán implementadas en Adoptet.

VII-A. Relación entre requerimientos e historias

Las historias de usuario permiten convertir los requerimientos generales en unidades de trabajo que pueden priorizarse y planificarse.

Por ejemplo, RF-07, RF-08 y RF-09 se materializan principalmente en HU-04, HU-05 y HU-06. Los requerimientos RF-10 y RF-11 se relacionan con HU-07, mientras que RF-13 y RF-14 se materializan principalmente en HU-12 y HU-13. Adicionalmente, RF-19 se materializa en HU-17 y RF-20 se materializa en HU-18.

VII-B. Casos de uso principales

Los principales casos de uso identificados para Adoptet son:

- **Adoptante:** registrarse, iniciar sesión, gestionar perfil, consultar mascotas, buscar, filtrar, ver detalle, solicitar adopción, consultar solicitudes y consultar seguimiento.
- **Administrador:** iniciar sesión, gestionar mascotas, gestionar usuarios, revisar solicitudes, actualizar estados y consultar historial.

VIII. CASOS DE USO

El diagrama general de casos de uso permite representar de manera visual los actores principales del sistema y las funcionalidades con las que interactúan.

Los actores se encuentran fuera del límite del sistema, mientras que los casos de uso se encuentran dentro del límite correspondiente a Adoptet.

- **CU-01 – Registrarse: Actor:** Adoptante. Crear una cuenta con datos personales, contacto y contraseña.
- **CU-02 – Iniciar sesión: Actor:** Adoptante, Administrador. Autenticarse en el sistema mediante credenciales válidas.
- **CU-03 – Recuperar contraseña: Actor:** Adoptante, Administrador. Restablecer la contraseña mediante un mecanismo de verificación.
- **CU-04 – Gestionar perfil: Actor:** Adoptante. Consultar y actualizar la información personal del adoptante.
- **CU-05 – Consultar mascotas: Actor:** Adoptante. Visualizar el listado de mascotas disponibles.
- **CU-06 – Buscar y filtrar mascotas: Actor:** Adoptante. Filtrar mascotas por especie, raza, sexo, edad, tamaño y disponibilidad.
- **CU-07 – Ver detalle de mascota: Actor:** Adoptante. Consultar la información detallada de una mascota seleccionada.
- **CU-08 – Solicitar adopción: Actor:** Adoptante. Registrar una solicitud de adopción asociada a una mascota disponible. Incluye la validación correspondiente.
- **CU-09 – Consultar solicitudes: Actor:** Adoptante. Consultar las solicitudes realizadas y su estado.
- **CU-10 – Consultar seguimiento: Actor:** Adoptante. Consultar los cambios relevantes del proceso de adopción.
- **CU-11 – Recibir notificaciones: Actor:** Adoptante. Recibir avisos sobre cambios de estado de sus solicitudes.
- **CU-12 – Gestionar mascotas: Actor:** Administrador. Registrar, actualizar y desactivar mascotas.
- **CU-13 – Gestionar usuarios: Actor:** Administrador. Consultar y administrar las cuentas de adoptantes.
- **CU-14 – Revisar solicitudes: Actor:** Administrador. Consultar las solicitudes de adopción recibidas.

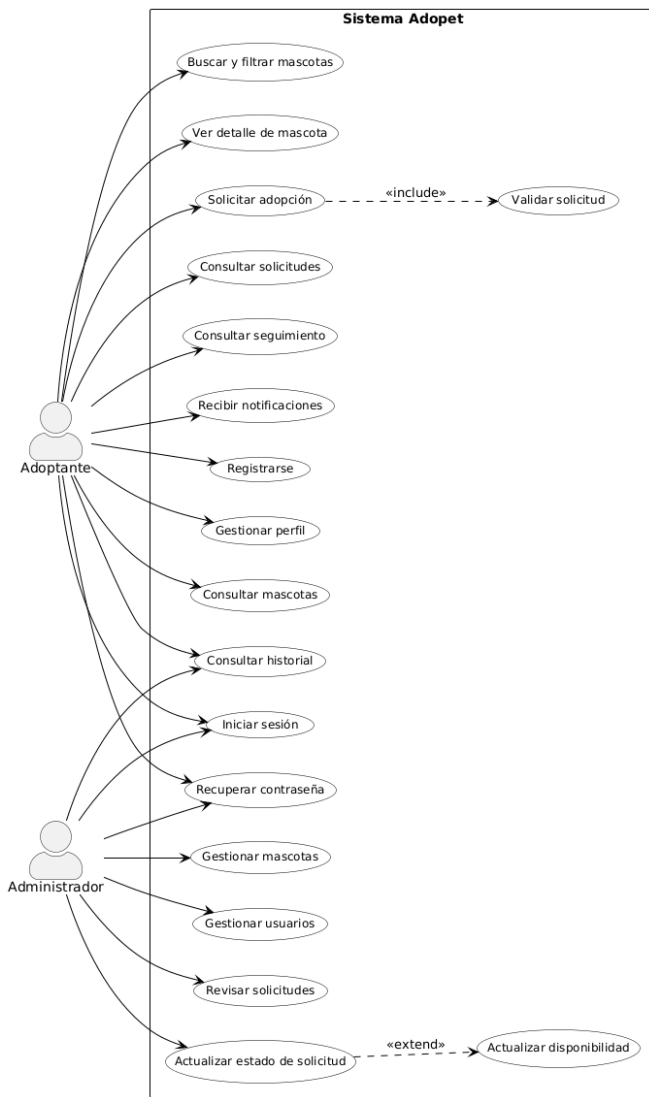


Figura 1. Diagrama general de casos de uso – Sistema Adopet.

- **CU-15 – Actualizar estado de solicitud:** Actor: Administrador. Establecer el estado de una solicitud. Este caso de uso extiende la actualización de disponibilidad de la mascota.
- **CU-16 – Consultar historial:** Actor: Adoptante, Administrador. Consultar el historial de solicitudes y adopciones.

IX. DIAGRAMAS DE CASOS DE USO

Los diagramas se modelaron en PlantUML con notación estándar (actor, elipse, límite del sistema, «include» y «extend»), equivalente a la empleada en Enterprise Architect. El código fuente se incluye en el Anexo A.

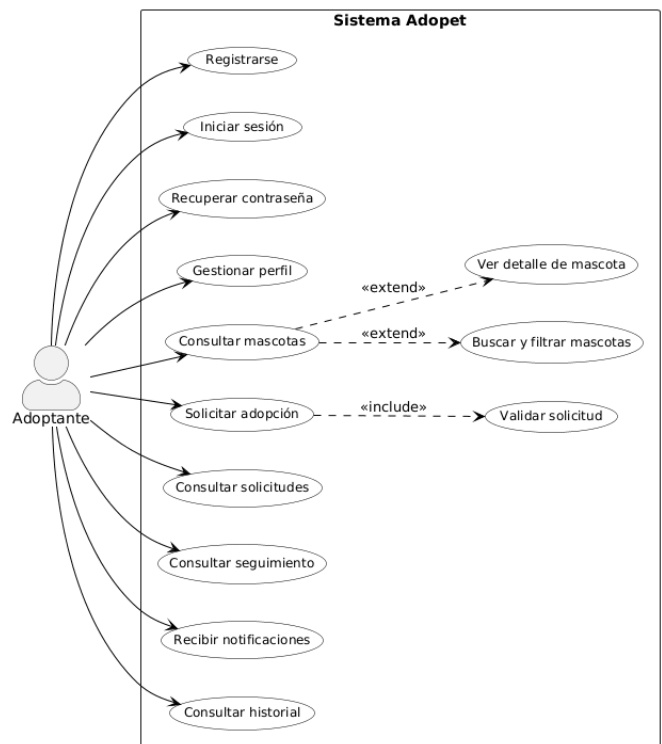


Figura 2. Diagrama de casos de uso – Actor Adoptante.

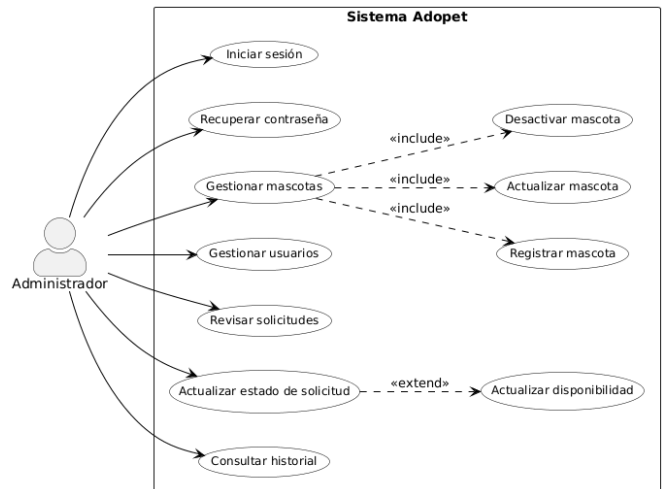


Figura 3. Diagrama de casos de uso – Actor Administrador.

X. HISTORIAS DE USUARIO

Las historias de usuario permiten expresar las necesidades funcionales desde la perspectiva de los actores del sistema. Estas representan funcionalidades que aportan valor al usuario y pueden ser priorizadas dentro del proceso de desarrollo ágil. La estructura utilizada es:

Como [rol], quiero [funcionalidad], para [beneficio].

■ HU-01 – Adoptante – Alta

Como adoptante, quiero registrarme para crear una cuenta

y poder realizar solicitudes.

- **HU-02 – Adoptante – Alta**

Como adoptante, quiero iniciar sesión para acceder de forma segura a mis funciones.

- **HU-03 – Adoptante – Media**

Como adoptante, quiero gestionar mi perfil para mantener actualizados mis datos.

- **HU-04 – Adoptante – Alta**

Como adoptante, quiero consultar mascotas disponibles para encontrar una que pueda adoptar.

- **HU-05 – Adoptante – Alta**

Como adoptante, quiero buscar y filtrar mascotas para encontrar opciones según mis preferencias.

- **HU-06 – Adoptante – Alta**

Como adoptante, quiero ver el detalle de una mascota para conocer sus características antes de solicitarla.

- **HU-07 – Adoptante – Alta**

Como adoptante, quiero enviar una solicitud de adopción para iniciar formalmente el proceso.

- **HU-08 – Adoptante – Alta**

Como adoptante, quiero consultar mis solicitudes para conocer su estado.

- **HU-09 – Adoptante – Media**

Como adoptante, quiero consultar el seguimiento para conocer los cambios de mi proceso.

- **HU-10 – Administrador – Alta**

Como administrador, quiero registrar mascotas para publicar animales disponibles.

- **HU-11 – Administrador – Alta**

Como administrador, quiero actualizar o desactivar mascotas para mantener información vigente.

- **HU-12 – Administrador – Alta**

Como administrador, quiero revisar solicitudes para evaluar los procesos de adopción.

- **HU-13 – Administrador – Alta**

Como administrador, quiero aprobar o rechazar solicitudes para tomar una decisión sobre cada proceso.

- **HU-14 – Administrador – Alta**

Como administrador, quiero actualizar estados y observaciones para mantener el seguimiento del proceso.

- **HU-15 – Administrador – Media**

Como administrador, quiero consultar el historial para conocer las adopciones y solicitudes anteriores.

- **HU-16 – Administrador – Media**

Como administrador, quiero gestionar usuarios para mantener control sobre las cuentas registradas.

- **HU-17 – Adoptante / Administrador – Media**

Como usuario del sistema, quiero recuperar mi contraseña para poder acceder nuevamente en caso de olvidarla.

- **HU-18 – Adoptante – Media**

Como adoptante, quiero recibir notificaciones para enterarme oportunamente de los cambios de estado de mis solicitudes.

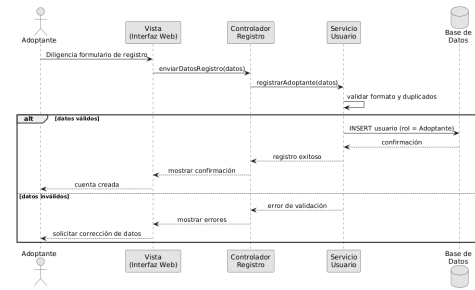


Figura 4. Diagrama de secuencia – CU-01 Registrarse.

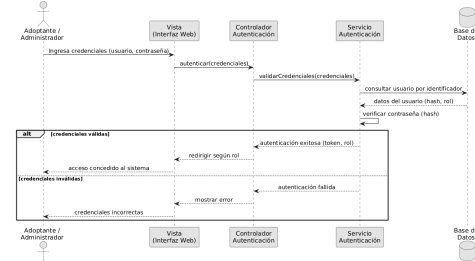


Figura 5. Diagrama de secuencia – CU-02 Iniciar sesión.

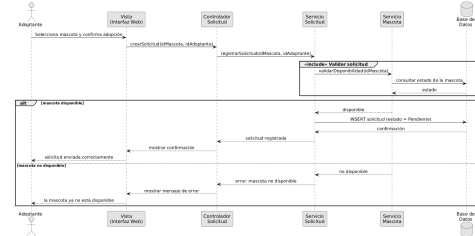


Figura 6. Diagrama de secuencia – CU-08 Solicitar adopción (incluye CU-15).

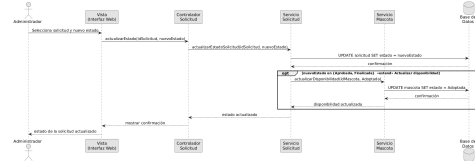


Figura 7. Diagrama de secuencia – CU-15 Actualizar estado de solicitud (extiende actualizar disponibilidad).

XI. DIAGRAMAS DE SECUENCIA

Los diagramas de secuencia modelan la interacción entre el actor, la capa de vista, el controlador, la capa de servicio y la base de datos, conforme a la arquitectura por capas (MVC) definida para Adoptet. Se priorizaron los flujos principales de cada actor. El código fuente se incluye en el Anexo A.

XII. DIAGRAMA DE CONTEXTO

El diagrama de contexto (nivel 0) delimita el alcance de Adoptet como un único proceso central, representado mediante la notación clásica de círculo relleno, que intercambia flujos de datos individuales con las entidades externas del sistema: los

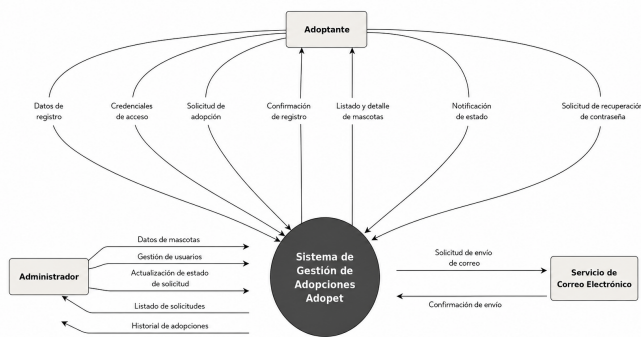


Figura 8. Diagrama de contexto (nivel 0) – Sistema Adopet.

dos actores (Adoptante, Administrador) y el servicio de correo electrónico requerido por RF-19 (recuperación de contraseña) y RF-20 (notificaciones). El código fuente se incluye en el Anexo A.

XII-A. Criterios de aceptación

Los criterios de aceptación permiten determinar cuándo una historia de usuario cumple las condiciones establecidas para considerarse funcionalmente terminada.

■ HU-01 – Registro

El correo no debe estar registrado previamente; los campos obligatorios deben validarse; la contraseña debe almacenarse de forma segura.

■ HU-04 – Consulta de mascotas

Solo deben mostrarse mascotas disponibles; cada resultado debe permitir acceder al detalle.

■ HU-07 – Solicitud

El usuario debe estar autenticado; la mascota debe estar disponible; la solicitud debe quedar registrada con fecha y estado inicial *Pendiente*.

■ HU-08 – Consulta de solicitudes

El adoptante solo debe visualizar sus propias solicitudes y su estado actual.

■ HU-10 – Registro de mascotas

El administrador debe ingresar los datos obligatorios; la mascota debe quedar disponible o en el estado seleccionado.

■ HU-12 – Revisión

El administrador debe poder listar solicitudes, consultar sus datos y abrir el detalle.

■ HU-13 – Decisión

Solo un administrador puede aprobar o rechazar; el cambio debe quedar registrado y reflejarse en el seguimiento.

■ HU-14 – Seguimiento

Cada cambio debe conservar fecha, estado y observación; el historial debe poder consultarse posteriormente.

■ HU-17 – Recuperación de contraseña

El sistema debe verificar la identidad del usuario mediante el mecanismo de verificación definido antes de permitir el restablecimiento; la nueva contraseña debe cumplir las

políticas de seguridad vigentes y almacenarse mediante hash seguro (RNF-03).

■ HU-18 – Notificaciones

El adoptante debe recibir una notificación ante cada cambio relevante de estado de una solicitud propia; la notificación debe reflejar el estado actualizado y la fecha del cambio.

XIII. METODOLOGÍAS ÁGILES

El desarrollo de Adopet se plantea bajo un enfoque ágil, considerando metodologías y marcos de trabajo que permitan realizar entregas incrementales, obtener retroalimentación continua y adaptarse a cambios en los requerimientos.

El Manifiesto para el Desarrollo Ágil de Software establece como valores principales la importancia de los individuos e interacciones, el software funcionando, la colaboración con el cliente y la respuesta al cambio [2].

Para seleccionar un enfoque apropiado para Adopet se analizaron tres alternativas:

1. Scrum.
2. Kanban.
3. Extreme Programming (XP).

XIII-A. Scrum

Scrum es un marco de trabajo ligero orientado a generar valor mediante soluciones adaptativas para problemas complejos.

Organiza el trabajo alrededor de Sprints, Product Backlog, Sprint Backlog e Incrementos, junto con eventos orientados a la inspección y adaptación [3].

El Scrum Team contempla tres responsabilidades principales:

- **Product Owner.**
- **Scrum Master.**
- **Developers.**

Scrum resulta especialmente útil para Adopet porque permite dividir el desarrollo en incrementos funcionales y priorizar las historias de usuario según el valor que aporten al producto.

XIII-B. Kanban

Kanban es un enfoque orientado a optimizar el flujo de valor mediante un sistema visual y basado en el flujo de trabajo.

Su guía enfatiza la definición y visualización del flujo de trabajo, la gestión de los elementos en progreso y la mejora continua del flujo [4].

Para Adopet, Kanban podría utilizarse mediante un tablero con estados como:

Pendiente → En desarrollo → En pruebas → Terminado

Su principal ventaja para el proyecto sería la facilidad para visualizar las tareas y detectar elementos pendientes o bloqueados.

XIII-C. *Extreme Programming (XP)*

Extreme Programming, conocida como XP, es un enfoque ágil que pone especial énfasis en las prácticas de ingeniería de software.

Entre sus principales prácticas se encuentran:

- Historias de usuario.
- Programación en parejas.
- Desarrollo guiado por pruebas.
- Integración continua.
- Diseño incremental.
- Refactorización.

Estas prácticas buscan incrementar la calidad técnica del software y obtener retroalimentación frecuente

Para un equipo de tres integrantes, algunas de estas prácticas pueden resultar especialmente útiles, como la programación en parejas y la revisión mutua del código.

XIII-D. *Comparación de metodologías*

■ **Organización**

- **Scrum:** Sprints, roles, eventos y artefactos.
- **Kanban:** Flujo continuo y tablero visual.
- **XP:** Iteraciones cortas y prácticas técnicas.

■ **Planificación**

- **Scrum:** Product Backlog y Sprint Backlog.
- **Kanban:** Priorización y flujo de trabajo.
- **XP:** Historias, ciclos cortos y planificación continua.

■ **Control**

- **Scrum:** Inspección y adaptación por Sprint.
- **Kanban:** Métricas y mejora del flujo.
- **XP:** Pruebas, integración y retroalimentación frecuente.

■ **Fortaleza para Adopet**

- **Scrum:** Facilita planificar un proyecto académico por entregas.
- **Kanban:** Es simple para visualizar tareas de un equipo pequeño.
- **XP:** Es fuerte para elevar la calidad técnica del código.

■ **Limitación**

- **Scrum:** Requiere disciplina con eventos y responsabilidades.
- **Kanban:** Tiene menos estructura temporal para una entrega académica.
- **XP:** Su conjunto completo de prácticas puede aumentar la exigencia técnica.

XIII-E. *Metodología seleccionada*

Se selecciona **Scrum** como marco principal para el desarrollo de Adopet.

La elección se fundamenta en que el proyecto puede dividirse en incrementos funcionales, requiere priorización de historias de usuario y se beneficia de ciclos cortos de planificación, revisión y mejora.

Kanban se considera una alternativa útil para visualizar el flujo de trabajo, mientras que XP aporta prácticas técnicas orientadas a la calidad del software.

Sin embargo, para el contexto académico de Adopet y teniendo en cuenta que el equipo está conformado por tres integrantes, Scrum proporciona una estructura clara para organizar, priorizar y realizar seguimiento al desarrollo.

La selección de Scrum no impide utilizar prácticas de los otros enfoques. Por ejemplo, el equipo podrá utilizar un tablero visual inspirado en Kanban y prácticas de calidad provenientes de XP.

XIV. IMPLEMENTACIÓN DE SCRUM EN ADOPET

XIV-A. *Adaptación al equipo de tres integrantes*

El equipo de Adopet está conformado por tres personas. Para mantener las responsabilidades definidas por Scrum y adaptarlas al contexto académico, se propone la siguiente distribución:

- **Julián Darío Romero Buitrago:** Product Owner y Developer.

Será responsable de priorizar el Product Backlog, representar las necesidades del producto, realizar el levantamiento y documentación de requisitos, participar en el diseño de la base de datos y desarrollar servicios y componentes de acceso a datos.

- **Juan Esteban Moreno Durán:** Scrum Master y Developer.

Será responsable de facilitar la aplicación del marco Scrum, ayudar a remover impedimentos, participar en el análisis de casos de uso, diseñar la interfaz de usuario y desarrollar los controladores del sistema.

- **Yuber Alejandro Bohorquez Roa:** Developer.

Será responsable de participar en la integración del sistema, implementar funcionalidades relacionadas con las solicitudes de adopción y realizar las pruebas necesarias para verificar el correcto funcionamiento del sistema.

- **Equipo de desarrollo:** Los tres integrantes participarán como Developers y colaborarán en la creación de incrementos funcionales que cumplan con la Definition of Done. Las responsabilidades podrán intercambiarse o compartirse según las necesidades de cada Sprint.

Esta distribución constituye una adaptación académica para un equipo de tres integrantes. Las responsabilidades de Scrum se mantienen aunque un integrante pueda asumir más de una función.

XIV-B. *Product Goal*

Desarrollar Adopet, un sistema web que permita gestionar de manera centralizada mascotas, adoptantes, solicitudes y seguimiento, facilitando un proceso de adopción organizado y trazable.

XIV-C. *Product Backlog*

El Product Backlog contiene las funcionalidades que deberán desarrollarse y se encuentra organizado según su prioridad y dependencias.

- **HU-01 – Registro – Prioridad: Alta – Sprint: 1**

Trabajo principal: Registro, validaciones y persistencia.

- **HU-02 – Inicio de sesión – Prioridad: Alta – Sprint: 1**
Trabajo principal: Autenticación y control de roles.
- **HU-03 – Perfil – Prioridad: Media – Sprint: 1**
Trabajo principal: Consulta y edición de datos.
- **HU-17 – Recuperación de contraseña – Prioridad: Media – Sprint: 1**
Trabajo principal: Mecanismo de verificación y restablecimiento de contraseña.
- **HU-10 – Mascotas – Prioridad: Alta – Sprint: 1–2**
Trabajo principal: CRUD y estados de disponibilidad.
- **HU-04 – Consulta – Prioridad: Alta – Sprint: 2**
Trabajo principal: Catálogo de mascotas disponibles.
- **HU-05 – Búsqueda – Prioridad: Alta – Sprint: 2**
Trabajo principal: Filtros de consulta.
- **HU-06 – Detalle – Prioridad: Alta – Sprint: 2**
Trabajo principal: Ficha detallada de mascota.
- **HU-07 – Solicitud – Prioridad: Alta – Sprint: 3**
Trabajo principal: Registro y validación de solicitudes.
- **HU-08 – Solicitudes – Prioridad: Alta – Sprint: 3**
Trabajo principal: Consulta del estado por adoptante.
- **HU-12 – Revisión – Prioridad: Alta – Sprint: 3**
Trabajo principal: Panel de solicitudes para administrador.
- **HU-13 – Decisión – Prioridad: Alta – Sprint: 3**
Trabajo principal: Aprobación y rechazo.
- **HU-14 – Seguimiento – Prioridad: Alta – Sprint: 4**
Trabajo principal: Estados, observaciones e historial.
- **HU-09 – Seguimiento adoptante – Prioridad: Media – Sprint: 4**
Trabajo principal: Consulta de cambios del proceso.
- **HU-18 – Notificaciones – Prioridad: Media – Sprint: 4**
Trabajo principal: Envío de notificaciones ante cambios de estado.
- **HU-15 – Historial – Prioridad: Media – Sprint: 4**
Trabajo principal: Consulta de procesos anteriores.
- **HU-16 – Usuarios – Prioridad: Media – Sprint: 4**
Trabajo principal: Gestión administrativa de usuarios.

XIV-D. *Sprints propuestos*

Se proponen cuatro Sprints de dos semanas cada uno.

Scrum establece que los Sprints tienen una duración fija de un mes o menos. Para Adoptet se seleccionan dos semanas con el objetivo de obtener ciclos cortos de inspección y adaptación [3].

- **Sprint 1 – Base del sistema**
Historias principales: HU-01, HU-02, HU-03, HU-10, HU-17.
Incremento esperado: Usuarios, autenticación, roles, recuperación de contraseña y gestión básica de mascotas.
- **Sprint 2 – Exploración de mascotas**
Historias principales: HU-04, HU-05, HU-06.
Incremento esperado: Catálogo funcional, búsqueda, filtros y detalle.
- **Sprint 3 – Solicitudes**
Historias principales: HU-07, HU-08, HU-12, HU-13.

Incremento esperado: Registro, consulta, revisión y decisión sobre solicitudes.

- **Sprint 4 – Seguimiento y cierre**
Historias principales: HU-09, HU-14, HU-15, HU-16, HU-18.
Incremento esperado: Seguimiento, notificaciones, historial, gestión de usuarios y estabilización.

XIV-E. *Eventos Scrum*

- **Sprint Planning:** al inicio de cada Sprint se define el Sprint Goal, se seleccionan historias y se descompone el trabajo.
- **Daily Scrum:** reunión breve diaria para inspeccionar el progreso, identificar impedimentos y ajustar el plan. La guía oficial establece una duración de 15 minutos para este evento [3].
- **Sprint Review:** al finalizar cada Sprint se revisa el incremento funcional y se identifican cambios necesarios en el Product Backlog.
- **Sprint Retrospective:** se analiza el proceso de trabajo, lo que funcionó, los problemas encontrados y las acciones de mejora.
- **Backlog Refinement:** actividad continua para aclarar, dividir y ordenar historias antes de incorporarlas a un Sprint.

XIV-F. *Artefactos Scrum*

- **Product Backlog:** lista ordenada de necesidades y funcionalidades del producto.
- **Sprint Backlog:** historias seleccionadas para el Sprint, objetivo y plan de trabajo.
- **Incremento:** versión funcional del producto que cumple la Definition of Done.

XIV-G. *Definition of Done*

Una historia de usuario se considerará terminada cuando:

- El código esté implementado y organizado en la arquitectura definida.
- Se hayan realizado las pruebas funcionales correspondientes.
- Los criterios de aceptación se cumplan.
- La funcionalidad esté integrada con los componentes necesarios.
- No existan errores críticos conocidos que impidan su uso.
- La documentación técnica necesaria esté actualizada.
- El cambio haya sido revisado por el otro integrante.

XV. ARQUITECTURA DEL SOFTWARE

XV-A. *Arquitectura propuesta*

Adoptet utilizará una **arquitectura por capas de tres niveles**, complementada con el patrón Modelo–Vista–Controlador (MVC).

Las capas principales son:

1. Capa de presentación.
2. Capa de lógica de negocio.
3. Capa de acceso a datos.

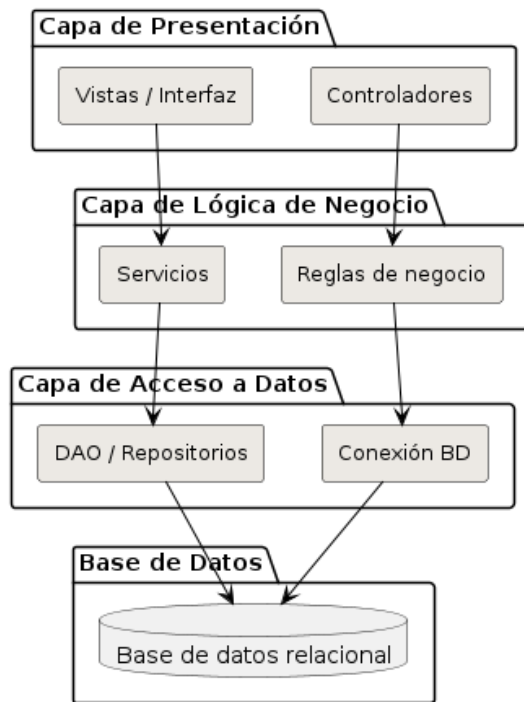


Figura 9. Arquitectura por capas de tres niveles propuesta para Adopet.

La base de datos constituye el componente de persistencia.

Esta arquitectura permite separar las responsabilidades del sistema y facilita el mantenimiento y evolución del software.

XV-B. Diagrama de arquitectura

La Figura 9 representa la arquitectura por capas de tres niveles propuesta para Adopet.

XV-C. Capa de presentación

Esta capa será responsable de la interacción directa con los usuarios del sistema.

Comprenderá las vistas, formularios, elementos de navegación y controladores necesarios para recibir las solicitudes del usuario y dirigirlos hacia la lógica correspondiente.

No deberá contener directamente reglas de negocio ni consultas SQL.

XV-D. Capa de lógica de negocio

Esta capa implementará los servicios y reglas de negocio del sistema.

Será responsable de realizar validaciones y controlar operaciones como la disponibilidad de mascotas, permisos de los usuarios y transiciones de estados de las solicitudes.

XV-E. Capa de acceso a datos

Esta capa gestionará la comunicación con la base de datos mediante DAO, repositorios y componentes de conexión.

Su objetivo será aislar la lógica de persistencia del resto de la aplicación.

XV-F. Patrón Modelo–Vista–Controlador

El patrón MVC se utilizará como complemento de la arquitectura por capas.

- **Modelo:** representará las entidades y datos del sistema.
- **Vista:** gestionará la interfaz presentada al usuario.
- **Controlador:** recibirá las solicitudes y coordinará las operaciones correspondientes.

La utilización conjunta de MVC y la arquitectura por capas permitirá mantener una separación clara de responsabilidades.

XVI. MODELO CONCEPTUAL DEL SISTEMA

XVI-A. Entidades principales

Las principales entidades identificadas para Adopet son:

- **Usuario:** id, nombre, documento, correo, teléfono, dirección, contraseña, rol y estado.
- **Mascota:** id, nombre, especie, raza, sexo, edad, tamaño, descripción, estado de salud, imagen y estado.
- **SolicitudAdopcion:** id, fecha, estado, observaciones y fecha de actualización.
- **Seguimiento:** id, fecha, estado y observaciones.

XVI-B. Relaciones

Un adoptante puede registrar varias solicitudes.

Cada solicitud se encuentra asociada con una mascota.

Una solicitud puede tener múltiples registros de seguimiento.

El rol del usuario determina los permisos y funcionalidades a las que puede acceder dentro del sistema.

XVI-C. Diagrama de clases

La Figura 10 presenta el diagrama de clases del sistema Adopet, modelado en notación UML. Se representa a *Usuario* como una clase abstracta de la cual heredan *Adoptante* y *Administrador*, cada uno con las operaciones propias de su rol. Las clases *Mascota*, *SolicitudAdopcion* y *Seguimiento* concentran los atributos y comportamientos identificados en el modelo conceptual, mientras que las enumeraciones *RolUsuario*, *EstadoMascota* y *EstadoSolicitud* restringen los valores permitidos para los atributos de estado correspondientes.

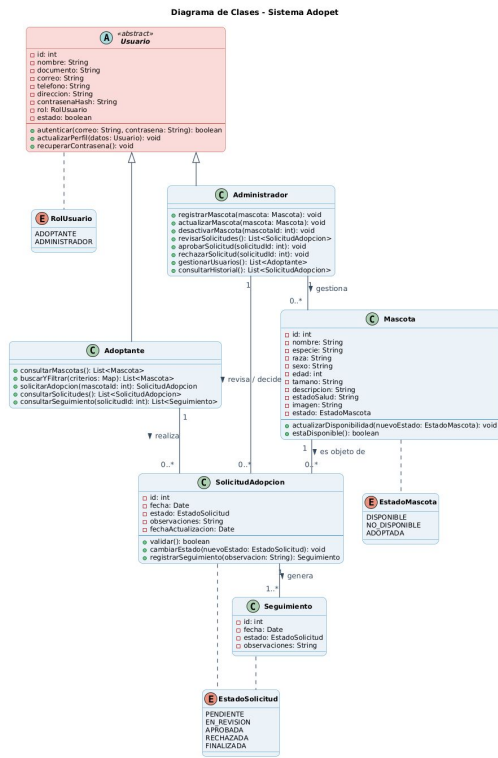


Figura 10. Diagrama de clases – Sistema Adoptet.

XVII. DIAGRAMA DE LA BASE DE DATOS

A partir del modelo conceptual y del diagrama de clases se deriva el modelo relacional que soportará la persistencia de Adoptet. Cada clase principal del dominio se traduce en una tabla, y las relaciones de asociación se implementan mediante llaves foráneas.

La tabla *usuarios* almacena tanto adoptantes como administradores, diferenciados mediante el atributo `rol`; esta decisión evita duplicar los atributos comunes de identificación y contacto que comparten ambos tipos de usuario. La tabla *mascotas* conserva la información publicada por los administradores. La tabla *solicitudes_adopcion* materializa la relación muchos a uno con *usuarios* (a través de `adoptante_id`) y con *mascotas* (a través de `mascota_id`), conforme a la regla de negocio RN-04. Finalmente, la tabla *seguimientos* conserva el historial de cambios de estado de cada solicitud, en una relación uno a muchos con *solicitudes_adopcion*, dando soporte a RF-15 y RF-17.

La Figura 11 presenta el modelo entidad-relación resultante.

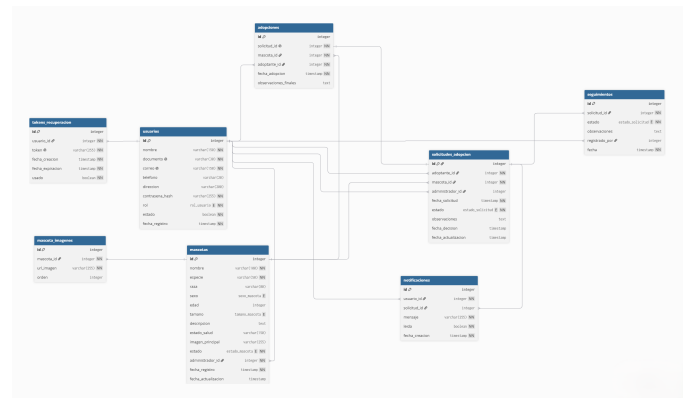


Figura 11. Diagrama entidad-relación – Base de datos de Adoptet.

XVIII. ESTRUCTURA PROPUESTA DEL SOFTWARE

La estructura propuesta refleja la separación de responsabilidades establecida mediante la arquitectura por capas.

```
Adoptet/
|
|-- presentation/
|   |-- views/
|   '-- controllers/
|
|-- business/
|   |-- services/
|   '-- models/
|
|-- data/
|   |-- dao/
|   '-- connection/
|
|-- resources/
|   '-- configuration/
```

```
|
|-- database/
    |-- scripts/
```

Esta estructura podrá adaptarse al lenguaje y tecnologías que sean seleccionados durante la etapa de implementación.

XIX. JUSTIFICACIÓN DE LA ARQUITECTURA

La separación por capas reduce el acoplamiento entre los componentes, facilita las pruebas y permite modificar una parte del sistema sin afectar directamente las demás.

La arquitectura también favorece el trabajo paralelo entre los tres integrantes y permite incorporar posteriormente nuevos módulos.

Entre las posibles funcionalidades futuras se encuentran:

- Notificaciones.
- Citas.
- Seguimiento veterinario.
- Reportes.
- Estadísticas.

De esta manera, la arquitectura propuesta proporciona una base flexible para el crecimiento de Adoptet.

XX. DIAGRAMA DE FLUJO DEL PROCESO

Con el fin de complementar la vista estructural aportada por el diagrama de clases y el modelo de base de datos, se elaboró un diagrama de flujo que describe el comportamiento dinámico del proceso de adopción de extremo a extremo, integrando el flujo del Adoptante y del Administrador a partir de un punto común de autenticación.

El proceso inicia con la autenticación del usuario según su *RolUsuario*. A partir de allí se derivan dos flujos paralelos: el flujo del Adoptante, en el que se consulta y filtra el catálogo de mascotas, se crea una solicitud de adopción y se valida su disponibilidad (RN-03); y el flujo del Administrador, en el que se gestionan las mascotas y se revisan las solicitudes pendientes. Ambos flujos convergen en la decisión de aprobación o rechazo de la solicitud (RF-14): si es rechazada, el proceso finaliza; si es aprobada, la mascota se actualiza al estado *ADOPTADA* (RF-16, RN-06) y se inicia un ciclo de seguimiento (RF-15) que se repite hasta que el proceso se marca como finalizado.

La Figura 12 presenta este diagrama.

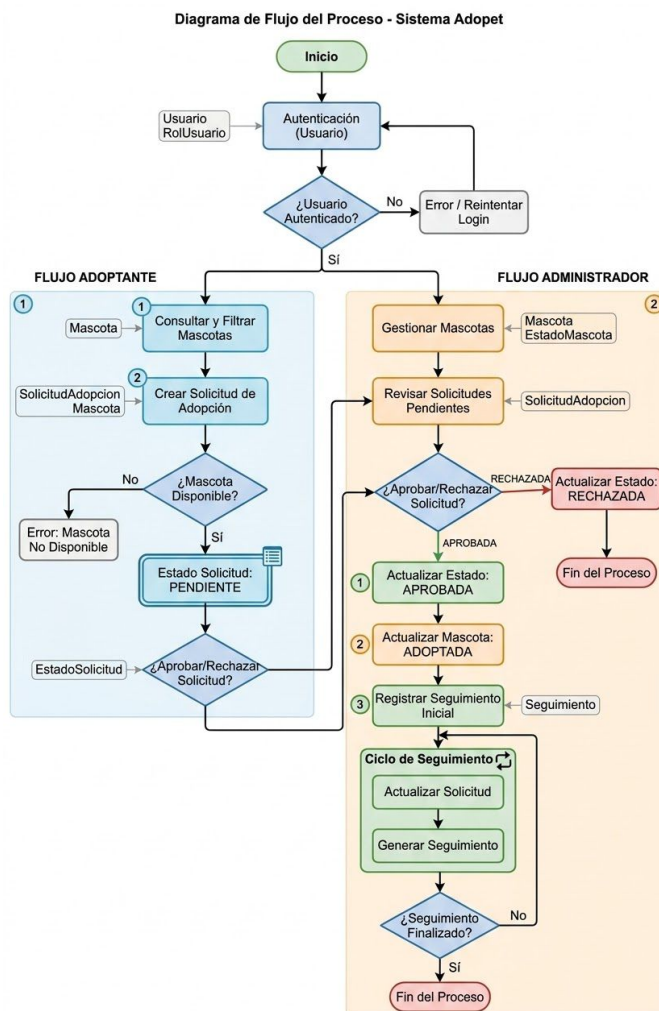


Figura 12. Diagrama de flujo del proceso de adopción – Sistema Adoptet.

XXI. TRAZABILIDAD ENTRE REQUERIMIENTOS, HISTORIAS Y ARQUITECTURA

La trazabilidad permite relacionar los requerimientos funcionales con las historias de usuario y con las capas principales de la arquitectura.

- **RF-01 – HU-01**
Funcionalidad: Registro de adoptante.
Capa principal: Presentación / Negocio / Datos.
- **RF-02 – HU-02**
Funcionalidad: Autenticación y roles.
Capa principal: Presentación / Negocio.
- **RF-04-06 – HU-10, HU-11**
Funcionalidad: Gestión de mascotas.
Capa principal: Negocio / Datos.
- **RF-07-09 – HU-04-06**
Funcionalidad: Consulta y detalle de mascotas.
Capa principal: Presentación / Datos.
- **RF-10-11 – HU-07**
Funcionalidad: Solicitud y validación.
Capa principal: Negocio / Datos.

■ **RF-12 – HU-08**

Funcionalidad: Consulta de solicitudes.

Capa principal: Presentación / Datos.

■ **RF-13-14 – HU-12, HU-13**

Funcionalidad: Revisión y decisión.

Capa principal: Negocio / Datos.

■ **RF-15-17 – HU-09, HU-14, HU-15**

Funcionalidad: Seguimiento e historial.

Capa principal: Negocio / Datos.

■ **RF-18 – HU-16**

Funcionalidad: Gestión de usuarios.

Capa principal: Presentación / Negocio / Datos.

■ **RF-19 – HU-17**

Funcionalidad: Recuperación de contraseña.

Capa principal: Presentación / Negocio / Datos.

■ **RF-20 – HU-18**

Funcionalidad: Notificaciones al adoptante.

Capa principal: Negocio / Presentación.

XXII. DISTRIBUCIÓN DEL TRABAJO

XXII-A. *Julián Darío Romero Buitrago – 20232020240*

Participación propuesta:

- Product Owner y Developer.
- Levantamiento y documentación de requisitos.
- Diseño de base de datos.
- Desarrollo de servicios y acceso a datos.

XXII-B. *Juan Esteban Moreno Durán – 20232020107*

Participación propuesta:

- Scrum Master y Developer.
- Análisis de casos de uso.
- Diseño de interfaz.
- Desarrollo de controladores.

XXII-C. *Yuber Alejandro Bohorquez Roa – 20231020195*

Participación propuesta:

- Integración del sistema.
- Implementación de funcionalidades relacionadas con las solicitudes.
- Pruebas del sistema.

Las responsabilidades podrán intercambiarse o compartirse según las necesidades de cada Sprint.

REFERENCIAS

- [1] ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*, ISO/IEC/IEEE, 2018.
- [2] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, *Manifesto for Agile Software Development*, 2001. [En línea]. Disponible: <https://agilemanifesto.org>
- [3] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, Scrum.org, 2020.
- [4] Kanban University, *The Kanban Guide*, 2025. [En línea]. Disponible: <https://kanbanguide.org>